

ASTEROIDS-LITE & LAUNCHER MENU

Vector ships, screen wraparound, splitting rocks — wired up under a process-based menu.

From a bouncing ball to a game of three

Week 2 stacked one pygame skill on the next. Today everything from the week shows up at once — vectors do the physics, a list manages the rocks, and a launcher knits the three games together.

Day 8

Window, loop,
event queue, clock

Day 9

Sprite + Group,
the OOP version

Day 10

Rect / circle
collisions

Day 11

Sound + frame
animation

Day 12

Scene state
machine

Day 13

Pong — full
end-to-end game

Day 14

Asteroids-Lite
+ launcher

BEFORE · yesterday

Pong runs on its own. Brick Breaker runs on its own. To play either, you cd into the right folder and run a separate command. Nothing ties them together — just three loose scripts.

AFTER · today

menu.py opens, three buttons sit on screen. Click one → that game runs in its own process. Press M in-game → back to the menu. A third game (Asteroids-Lite) joins the collection.

Pointing where you want to go

A ship has a position, a velocity, and a facing angle. Rotation changes the angle; thrust adds acceleration along the direction the ship is currently pointing.

```
import math, pygame

class Ship:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y)
        self.vel = pygame.Vector2(0, 0)
        self.angle = 0 # degrees, 0 = up

    def update(self, keys, dt):
        if keys[pygame.K_LEFT]: self.angle -= 180 * dt
        if keys[pygame.K_RIGHT]: self.angle += 180 * dt
        if keys[pygame.K_UP]:
            rad = math.radians(self.angle - 90)
            thrust = pygame.Vector2(math.cos(rad), math.sin(rad))
            self.vel += thrust * 200 * dt
        self.pos += self.vel * dt
```

- ▶ **pygame.Vector2** Same idea as your Day-1 Vector3D — just 2D and built in. Supports +, *, .length, .normalize.
- ▶ **Angle → radians** math.cos / math.sin take radians, not degrees. radians() converts.
- ▶ **The -90 offset** Screen y grows downward, and we want 0° to mean 'up'. Subtracting 90° rotates the axis.
- ▶ **dt everywhere** Multiplying speeds by dt makes the ship behave the same at 30 or 144 FPS.

Off one edge, on the other

Asteroids space is a torus — leave the right edge, come back on the left. The modulo operator does this in one line per axis, no if-statements needed.

```
SCREEN_W, SCREEN_H = 800, 600
```

```
def wrap(pos, w, h):  
    pos.x %= w  
    pos.y %= h  
    return pos
```

```
# inside Ship.update, after moving pos:
```

```
self.pos = wrap(self.pos, SCREEN_W, SCREEN_H)
```

```
# same call for every asteroid and every bullet
```

```
for a in asteroids: a.pos = wrap(a.pos, SCREEN_W, SCREEN_H)
```

- ▶ **% is remainder** $x \% 800$ forces x into the range $[0, 800)$. One line, no branches.
- ▶ **Negative inputs work** In Python, $-5 \% 800 = 795$ (unlike C). So crossing the left edge wraps cleanly to the right.
- ▶ **Reuse everywhere** Same `wrap()` handles ships, rocks, and bullets — it doesn't care what object it's wrapping.
- ▶ **Visual cheat** Ships near an edge can be drawn twice (once on each side) to avoid pop-in. Optional polish.

A list of rocks that breaks into smaller rocks

All asteroids live in a Python list. When one is hit, remove it and (if it was big enough) drop two smaller asteroids back into the list at the same spot.

```
import math, random, pygame

class Asteroid:
    def __init__(self, pos, size=3):
        self.pos = pygame.Vector2(pos)
        ang = random.uniform(0, 360)
        rad = math.radians(ang)
        self.vel = pygame.Vector2(math.cos(rad), math.sin(rad)) * 60
        self.size = size          # 3=big, 2=mid, 1=small
        self.radius = size * 15

def split(a):
    if a.size > 1:
        return [Asteroid(a.pos, a.size - 1),
                Asteroid(a.pos, a.size - 1)]
    return []
```

- ▶ **List = your world** asteroids is a plain list. Iterate to draw, iterate to update, mutate to add/remove.
- ▶ **Random direction** Each rock picks its own angle on spawn — gives the field that scattered feel.
- ▶ **Size is the story** size 3 → 2 → 1 → gone. Radius and split logic both key off this one number.
- ▶ **split() returns a list** Empty list when the rock is already tiny. Then extend(): asteroids.extend(split(a)).

Short-lived projectiles, distance-based hits

Bullets are tiny objects with a lifetime. Each frame, step them forward, drop expired ones, and check every bullet against every asteroid using distance — circle-to-circle collision.

```
class Bullet:
    def __init__(self, pos, angle):
        rad = math.radians(angle - 90)
        self.pos = pygame.Vector2(pos)
        self.vel = pygame.Vector2(math.cos(rad), math.sin(rad)) * 400
        self.life = 1.0 # seconds before it dies

# every frame:
for b in bullets[:]: # copy → safe to remove
    b.pos += b.vel * dt
    b.life -= dt
    if b.life <= 0: bullets.remove(b)

for b in bullets[:]:
    for a in asteroids[:]:
        if b.pos.distance_to(a.pos) < a.radius:
            bullets.remove(b); asteroids.remove(a)
            asteroids.extend(split(a))
            break
```

- ▶ **Copy before removing** `list[:]` gives a shallow copy. Removing from the original while looping the copy keeps the iterator sane.
- ▶ **distance_to() built in** `Vector2` already computes $\sqrt{dx^2 + dy^2}$. Compare to `a.radius` — that's your hit-test.
- ▶ **break the inner loop** Once a bullet hits, it's gone — don't let it score against a second rock in the same frame.
- ▶ **Lifetime > screen check** Off-screen bullets wrap (modulo), so they'd never die. Time-to-live solves that cleanly.

Three buttons, three processes

The menu opens each game in its own process. A Button is just a `pygame.Rect` with a label and a function to call — when the click lands inside the rect, fire the function.

```
import subprocess, sys, os, pygame
```

```
def launch(filename):
    here = os.path.dirname(os.path.abspath(__file__))
    path = os.path.join(here, filename)
    subprocess.run([sys.executable, path])
    # blocks until the game closes, then we're back
```

```
class Button:
    def __init__(self, label, rect, action):
        self.label, self.rect, self.action = label, pygame.Rect(rect), action

    def click(self, pos):
        if self.rect.collidepoint(pos): self.action()

# wired up:
buttons = [Button("Asteroids Lite", (300, 400, 200, 60), lambda: launch("asteroids_lite.py")), ...]
```

- ▶ **sys.executable** Uses the same Python that's running the menu — no env mismatch, no PATH gotchas.
- ▶ **subprocess.run blocks** The menu pauses while the game runs. When the game quits, control returns and the loop resumes.
- ▶ **Each game = own process** If Asteroids crashes, the menu and other games are untouched. Clean isolation, free.
- ▶ **collidepoint(pos)** Rect has it built in. Pass the mouse pos from a `MOUSEBUTTONDOWN` event.
- ▶ **Action = a function** Storing the callable lets one Button class handle all three games. Lambdas keep wiring short.

Three games, three mechanics

Same engine (pygame), same architecture pattern, three completely different feel-spaces.

Game	Control	Core mechanic	Collision
Brick Breaker	Mouse paddle	Ball bounces, bricks vanish on hit	rect ↔ rect
Pong	W/S keys + AI right paddle	Two paddles, angle deflection on hit	rect ↔ rect
Asteroids Lite	Rotate (←/→) + thrust (↑)	Vectors, wraparound, splitting rocks	circle distance

All three sit behind the same menu — the launcher pattern doesn’t care what’s in each .py file.

 Vectors unify the three

Brick Breaker and Pong use rects because their shapes are rectangular. Asteroids uses circle distance because rocks are roughly round and rotation breaks rect alignment.

Ship the launcher + Asteroids-Lite

Two new files today. menu.py runs a 3-button selector and uses subprocess to launch each game. asteroids_lite.py is the new game — vector ship, wraparound, splitting rocks. Both Pong and Brick Breaker (already written) gain an M-key handler that returns to the menu.

STRUCTURE

menu.py + asteroids_lite.py
(brick_breaker.py and pong.py already exist
— just add M-to-menu)

All files in one folder.
menu.py uses sys.executable + os.path so it
works from any cwd.

CLASSES

Ship, Asteroid, Bullet, Button.

pygame.Vector2 for every position and
velocity.
asteroids and bullets are plain lists.
Button stores label, Rect, and an action
function.

BEHAVIOR

← / → rotate. ↑ thrust. SPACE fires.
M (in any game) → quit back to menu.

All objects wrap on screen edges.
Rocks split twice (size 3 → 2 → 1 → gone).
Start with 5 rocks; respawn ship on death.

Three traps you'll hit, two habits to keep

⚠ Working directory bites

`subprocess.run` uses the current `cwd` by default — and on macOS that often isn't your script folder. Build the absolute path with `os.path.dirname(os.path.abspath(__file__))` before launching.

⚠ Don't mutate while iterating

for `a` in `asteroids`: `asteroids.remove(a)` silently skips items. Loop over a copy: for `a` in `asteroids[:]`. Same trick for bullets, sprites, anything mutable.

💡 dt over frame-counting

Multiply every velocity, angular speed, and lifetime by `dt`. The game then behaves identically at 30 FPS on an old laptop and 144 FPS on a gaming monitor.

💡 Use `Vector2`'s methods

`.distance_to`, `.length`, `.normalize`, `.rotate` are all built in. Reaching for `math.sqrt` or `math.atan2` is usually a sign you're reimplementing something the library already has.

Week 2 closes. Week 3 starts tomorrow.

WEEK 3 · MAY 25

Comprehensions, generators, decorators, subprocess + hotkeys.

Project: CLI Autoclicker — global hotkeys + click-pattern record/replay. Builds directly on today's subprocess pattern.

THE LAUNCHER PATTERN

Reusable beyond games.

Same shape will hold any future tool collection: drop in a new .py, drop in a new Button. Don't touch the menu.

VECTORS COMPOUND

pygame.Vector2 is your Vector3D in 2D.

Same operators, same .normalize, same .length. The abstraction you built on Day 1-2 keeps paying off.

From OOP fundamentals to a process-launching game launcher — in 14 days.